Decision records

Short notes on decisions that shape the project: what was decided, why, and what follows from it.

1. Bill search

Decision. One search page. The user types a client name or a bill number and gets that client's bills as a list, newest first, with the bill date and bill number shown prominently.

Why. An accountancy firm can have hundreds of bills per client. Finding one by name or number is the common task.

Consequences.

- Results are paginated (20 per page) instead of loading everything.
- `bills` has indexes on `(bill_date DESC, id DESC)` and `client_id` so search and sorting stay fast.
- Two clients can share a name, so each result also shows the phone and city.
- Bill number is unique (`INV-000001` , from a database sequence); client name is only a search term.

2. Bills can be edited in v0.1

Decision. Any bill can be edited for now. This may change in a later version.

Why. It keeps v0.1 simple, and the firm can fix typos.

Consequences.

- Known risk: in real accounting, an issued bill is normally cancelled or corrected with a credit note.
- Safeguard: every bill stores `created_by` , `created_at` , `updated_by` and `updated_at` , so we can always see who changed it and when. Credit notes can be added later without losing data.

3. GST is built, but off by default

Decision. GST support exists and an admin turns it on with a toggle in organisation settings (with a rate).

Why. The firm does not charge GST yet, but will.

Consequences.

- Tax is **snapshotted on each bill** (`gst_rate_percent` , `gst_paise`). Changing the setting never alters old bills.
- Editing a bill recalculates using *that bill's* stored rate, not the current setting.
- GST uses integer maths on basis points and rounds half up.
- Settings are admin-only to change; every user can read them (the bill form needs the rate).
- Not built yet: CGST/SGST/IGST split, HSN/SAC codes, GSTR reports.

4. Admin creates staff accounts

Decision. No public sign-up. The admin creates accounts. Roles are `admin` and `staff`.

Why. This is an internal tool for one firm. Anyone on the internet must not be able to create an account.

Consequences.

- The first admin is created by a seed script (`npm run seed`) using values from `server/.env`.
- Staff are deactivated, not deleted, so old bills still show who created them.
- The user is loaded from the database on every request, so deactivation takes effect immediately.
- An admin cannot deactivate or demote their own account (prevents locking everyone out).
- Repeated failed logins are rate limited per username and IP address.

5. Money is stored as integer paise

Decision. Every amount is a whole number of paise in the database, API and server code.

Why. Floating point cannot represent values like 0.1 exactly (`0.1 + 0.2` is `0.30000000000000004`), which is unacceptable in billing.

Consequences.

- The API takes and returns `*_paise` fields; only the UI converts to rupees.
- The UI converts typed amounts by splitting the text, never by multiplying a float.

6. Soft delete for clients, services and staff

Decision. Records are flagged (`is_deleted` / `is_active`) instead of removed.

Why. Bills refer to clients, services and users. Removing them would break old bills or lose history.

Consequences.

- Deleted clients are hidden by default but can be shown and restored.
- A bill may keep belonging to a deleted client, but new bills cannot be created for one.

7. Windows installer: one office PC as the server, embedded database

Decision. A Setup.exe wizard (Inno Setup) installs the app on one PC as a Windows service. Other staff use it from a browser over the office network. The installer bundles Node.js and uses PGlite (Postgres compiled to WebAssembly) as the database, so nothing else has to be installed. It is built and published by GitHub Actions when a version tag is pushed. Windows only.

Why. The firm is small and non-technical. Installing PostgreSQL separately is the step most likely to go wrong.

Consequences.

- All data lives in `C:\ProgramData\BillingSoftware`, separate from the program, so upgrades never touch it.

- The embedded database allows only one process to open it, so only the service uses it. Backups are taken by the server itself (at startup and every 24 hours, last 14 kept), and `restore.cmd` is used to restore one.
- Backups on the same PC do not protect against that PC failing; the wizard tells the user to copy them elsewhere.
- The login secret is generated once on first start and stored in the data folder.
- The installer is not code-signed, so Windows shows an "unknown publisher" warning.
- The firewall rule covers private and domain networks only. The app uses plain HTTP and is meant for the office LAN, not the internet.
- Larger firms can still point the same server at a real PostgreSQL server with `DATABASE_URL`.

9. A readable folder per client, kept on the PC

Decision. The server writes a folder for every client (named after them) containing a details file, a JSON copy and an HTML copy of every bill. It is kept up to date automatically and rebuilt at startup.

Why. The firm wants its client records as ordinary files it can open, copy or send, independent of the software.

Consequences.

- It is a second copy, not the source of truth: the database is. The files are rewritten, so editing them has no effect.
- Folder names are made safe for Windows (illegal characters become `-`, reserved names and trailing dots are handled). Duplicate names get the client number added. The chosen folder is stored in `clients.folder_name`.
- Folders are only ever created, renamed or refreshed, never deleted. A bill that moves to another client, or whose date changes, has its old copy removed from the old place so the folder never shows stale data.
- File problems never block saving a bill; they are logged and shown to admins in Settings.
- Like the database backups, these folders live on the same PC, so the firm should copy them elsewhere regularly.

10. Financial years and service descriptions on bills

Decision. Bills are grouped by Indian financial year (1 April to 31 March), with a dropdown on the Bills page. A bill line carries a copy of the service's description.

Why. Accountants work and file by financial year. The description explains what the client is paying for.

Consequences.

- The FY is calculated from the bill date, not stored, so it can never disagree with the date.
- The description is copied onto the line when the bill is made (editable per bill), so changing the service later does not rewrite bills already issued, the same rule as the GST snapshot.

11. Logo and payment details on every bill

Decision. An admin sets the firm's logo and payment details (account holder, bank, account number, IFSC, branch, UPI ID, free-text instructions) in Settings. They print on every bill and in the client-folder copies. Anything left empty is not printed.

Why. Clients need to know where to pay.

Consequences.

- They are the firm's *current* details, shown on every bill (not frozen per bill like GST), because when the firm changes its bank account, bills that are still unpaid should show the new one. Saving them refreshes the client-folder copies.
- The logo is stored in the database (table `org_logo`), so it is included in the normal backups.
- Only PNG, JPEG and WebP are accepted, up to 1 MB, and the real file type is checked from the file's contents. SVG is refused because it can contain scripts. The browser shrinks large images before uploading.
- Only admins can change these; every logged-in user can view them. The logo is not public: it needs a login to fetch.
- The payment details are plain text typed by the admin. The software does not take payments or check the account.

12. Themes and design principles

Decision. The app has light, dark and "follow the device" themes, a token-based design system (all colours, shadows and radii are CSS variables) and follows the guidance in github.com/emilkowalski/skills.

Why. Staff use the software for hours a day, so comfort, clarity and not being slowed down matter more than flourish.

Consequences.

- **Motion is deliberately minimal.** Used-all-day actions (opening a list, typing, keyboard shortcuts) never animate. Only press feedback (160 ms, scale 0.97), colour changes (150 ms) and message fades (200 ms) move, with one strong ease-out curve and never `ease-in` or `transition: all`. Reduced-motion users get the fades without the movement.
- **A bill is a document.** It is always a white sheet (`.paper`), in dark mode too, because it prints on white and logos with dark text on a transparent background would vanish on a dark card.
- **Worst-case data is a test.** Long names (up to the real 150-character limit), emoji and non-Latin names, one-letter names, maximum-length descriptions and 12-item bills were loaded into a throwaway database and checked at 320 px wide, in both themes. Amounts and dates are never truncated; text wraps; badges never wrap.
- **The frontend and backend agree on limits.** Every field enforces its server limit as you type.
- **Bill totals have a ceiling.** A subtotal above ₹100 crore is refused, because quantity x price can otherwise exceed what can be added exactly.
- **Not verified:** the guidance says emulation cannot reproduce sticky hover, safe areas or the on-screen keyboard. Please also try the app on a real phone.

8. Plain SQL and a small database adapter

Decision. SQL migrations and hand-written queries (no ORM yet), behind a tiny `Db` interface.

Why. Learning goal: see exactly what the database is asked to do. The interface also lets the tests run on PGlite (Postgres in WebAssembly) with no database to install, while production uses the real `pg` driver.

Consequences.

- Migrations live in `server/migrations/` and run in order at startup.
- An ORM can be introduced later without changing how the app is structured.